IUT de Paris Rives de Seine – Département Informatique – BUT

R4.02 Qualité du Logiciel

TP 1

Thèmes : Test par les spécifications

Certains exercices sont tirés ou adaptés de [Aniche 22]

Exercice 1

Points ON, OFF = limites entre les partitions
Points IN et OUT = points dans la partition qui n'est pas ON ou OFF
Partition = cas possible (dont plantage ?)

Donnez les points ON, OFF, un point IN et un point OUT pour les conditions suivantes, X étant un entier. Lesquels de ces points faut-il tester ? Les valeurs limites

a) X <= 120 2 partitions : vrai ou faux,]-inf, 120] et]120, +inf[. Le point OFF est 121 (appartient à l'autre partition)
un point IN : -40 un point OUT : 5000

b) X == 10 3 partitions :]-inf, 10[, {10} et]10, +inf[on a deux points OFF, 9 et 11 et le point ON est 10
Un point IN : Aucun, un point OUT : -50 ou 150

3 partitions dans les égalités

Exercice 2

Un programme prend des commandes sur l'entrée standard et pour celles de la forme suivante cherche les occurrences d'un pattern (une chaine de caractères) dans un fichier textuel :

chercher "<pattern>" <nom de fichier>

Un testeur a défini les partitions suivantes :

- Taille du pattern : vide, un caractère, plusieurs caractères, plus grand que n'importe quelle ligne du fichier ou objet null pour une classe
- Guillemets autour du pattern : présents, non présents, incorrects
- Nom de fichier : absent, présent mais aucun fichier de ce nom, correct
- Occurrences dans le fichier : aucune, une, plus d'une
- Occurrences dans une ligne qui contient le pattern : une, plus d'une

Tests d'erreur à séparer du reste :
1. Pattern vide
2. Pattern + grand que la ligne
3. Pas de guillemets
4. Guillemets incorrect
5. Fichier absent
6. Fichier non existant

Le nombre de combinaisons de ces partitions étant trop élevé comment le réduire ?

On va isoler cas bizarres / anormaux, on les test séparément

Trouver les dépendances entre les cas (Si un fichier n'existe pas on ne testera pas les occurrences dans le fichier par exemple)

Exercice 3

La méthode ci-dessous, de la classe Dico<K key, V value>, place la valeur fournie dans un dictionnaire avec la clé fournie. Une IllegalArgumentException est levée si la clé est nulle. Retourne la précédente valeur de la clé s'il y en avait une, null sinon.

```
public V put(K key, V value);
```

Pour le résultat 2 cas : null ou pas null
Pour la clé 3 cas : null, déjà présente ou pas présente
Pour la valeur 2 cas : null ou pas null

A) Définissez les partitions sur les données correspondant à cette spécification

B) Définissez un test pour chaque partition et codez la classe Dico en vous basant sur la classe HashMap de Java qui ne lève pas autant d'exceptions.

Cas bizarre : Si l'ancienne value valait null alors la méthode renvoie null mais on ne sait pas si c'est parce qu'il n'y avait pas de valeur précédente ou si c'est parce que la valeur précédente était null

4 tests : Deux exceptions et deux cas de clé non null

1. Clé nulle
2. Valeur null
3. Clé absente (non null) et valeur non nulle
4. Clé déjà présente valeur non nulle

Exercice 4

Le programme FizzBuzz prend en entrée un nombre entier n et retourne une chaine composée de n suivi d'un point d'exclamation. Toutefois si n est un multiple de 3 le programme retourne « Fizz! ». Par ailleurs, si n est un multiple de 5 la réponse est « Buzz! ».

Codez le programme et testez-le après avoir partitionné l'espace des données.

1. Ni multiple de 3 ni multiple de 5
2. Multiple de 3
3. Multiple de 5
4. Multiple de 3 ET de 5 Mauvais partitionnement : on ne devrait pas avoir d'intersection (il faut demander au client)